ChatGPT

Outbreaks: Preempting Waterborne Disease Outbreaks with AI and Satellite Data

Tagline: *Preempting waterborne disease outbreaks with AI and satellite data.*

Earth is mostly covered by water, but not all water is safe to drink [1]. In fact, WHO reports that about **1.7 billion people use drinking water sources contaminated with feces** [2]. Contaminated water can spread diseases like cholera, typhoid and dysentery, causing roughly **505,000 diarrheal deaths each year** [3]. Right now, outbreaks are usually detected **after** people get sick, which makes the response slow and costly. We need an **early warning system** so that health teams can act *before* an outbreak strikes.

Multi-Source Data Fusion

To predict outbreaks early, we collect many types of data and **fuse** them into a unified dataset:

- **Satellite (Remote Sensing) data:** NASA/ESA satellites measure environmental conditions. For example, they track sea-surface temperature (SST), chlorophyll-a (algae blooms), rainfall and flooding. These factors affect water quality. Satellites can even map flooded areas over time, which often predict where cholera and other waterborne diseases may spread [4].
- **Climate & Weather data:** We use forecasts and historical records of rain, temperature and drought (e.g. from NOAA or NASA). Scientists have found that *warmer* waters lead to more bacteria outbreaks (like Vibrio infections) in coastal areas [5]. By including weather trends and ocean temperature, the model can anticipate conditions that favor pathogens.

- **Socio-economic data:** We include human factors such as **population density**, access to improved water sources, and sanitation quality (for example from WHO/JMP and local databases). Areas with high population and poor sanitation are more vulnerable.
- **Crowdsourced and local data:** This includes anonymous mobility data (to see how people move after floods) and reports from health clinics or community sensors. For instance, if many people report stomach illness in a village, that information can feed into the model.

When heavy rains cause flooding, sewage can mix with drinking water and trigger outbreaks [4]. Satellites can **map these flood zones** so we know which villages or towns are at risk. For example, researchers mapped flooding around India's Vembanad Lake to identify cholera "hotspots" [4]. In our system, we would regularly update flood and rainfall maps so that high-risk zones light up on our dashboard *before* people get sick.

Building the Predictive Model

With this fused dataset, we train an AI model to output a **risk score (0–100)** for any location and date. We start with **synthetic data** (fake but realistic) to build a proof-of-concept:

1. **Generate synthetic data:** Create a table of sample points with columns like `latitude`, `longitude`, `date`, and environmental features (e.g. precipitation, temperature, chlorophyll index, flood extent, population density, etc.). You can generate random values or use simple rules (e.g. "if rain > X and temperature > Y, then risk is higher"). Also include a "ground truth" column (e.g. 0 or 1, or a continuous risk) to simulate past outbreaks.
2. **Train the model:** Use a tree-based regressor on this data. For example, train **XGBoost** (an efficient gradient boosting library) to learn the mapping from features to risk. If XGBoost is not available, you can use Scikit-learn's `GradientBoostingRegressor`. Split your data into training and test sets. Fit the model on the training data.
3. **Evaluate & tune:** Check performance on the test set. Calculate metrics like MAE or RMSE. Look at **feature importances** to see which factors matter most. (For example, you might find heavy rainfall and high population density are top predictors.) Adjust parameters or features if needed.
4. **Score new points & map:** With the trained model, you can feed in any new `(lat, lon, date)` and get a **risk score** between 0 and 100. To visualize results, use Folium (a Python map library) to plot the risk on a map. For example, plot each location as a colored dot (red = high risk, green = low risk), or create a heatmap overlay. This produces an interactive risk map (saved as HTML) showing hotspots.

Web API and Dashboard

Next, wrap the model in a web service and interface for end users:

1. **Develop the REST API:** Use **Flask** (a simple Python web framework) to create endpoints. For instance:
2. `POST /score`: Accept a JSON object like `{"lat": 0.5, "lon": 32.5, "date": "2024-01-10"}` and return a JSON with the predicted risk score.
3. `POST /score/batch`: Accept either a JSON list of points or an uploaded CSV file of points, and return a list of scores. This allows batch processing.

4. `GET /export/csv`: Return all scored points (from a recent run) as a downloadable CSV file. *(You can also add* `GET /export/pdf` *if you want a PDF report, but CSV is usually enough.)*
5. **Build the web dashboard:** Create a simple front-end page (using Flask templates or a static page) that shows the Folium risk map. On this page, include:
6. A map view displaying the latest risk map (the HTML Folium map).
7. A form where a user can enter a location (lat/lon) and date to get an immediate risk score.
8. A file upload button where users can upload their own CSV of points to score. After upload, the page can display the new points on the map.
9. Optionally, a **notification** feature: e.g. if a location's score exceeds a threshold, the system could use an SMS API (like Twilio) or email to send an alert to local health workers with the location and risk level.

Example pseudo-steps for the Flask app:

```
pip install flask pandas xgboost folium
```

Then in `webapp.py`, set up routes using Flask. Within the `/score` route, parse the input JSON, call your model's `predict()`, and return the score.

Deployment and Demo

1. **Containerize with Docker:** Write a `Dockerfile` that installs the Python environment and your code. For example, start from `python:3.9-slim`, copy in your project files, run `pip install -r requirements.txt`, and set the entrypoint to run the Flask app on port 8001. Then a simple `docker-compose.yml` can define this service. With this, anyone can run:

   ```
   docker compose up --build
   ```

   and it will launch the app on `http://localhost:8001`. Containerization ensures it runs the same way on any machine.

2. **Static demo for GitHub Pages:** To share a demo without needing a server, pre-generate results (like `risk_map.html` and a sample dataset) and place them in a `docs/` folder. Create a simple HTML page in `docs/index.html` that embeds the precomputed map or data. Then enable GitHub Pages on the `main` branch with the `/docs` folder. Users can then browse a live demo at `<username>.github.io/<repo>`. (This static version won't run new predictions, but it shows the idea.)

Quick Start

Once implemented, you (or others) can run the system as follows:

- **Install dependencies** (in a virtualenv):

```
pip install -r requirements.txt
```

- **Generate initial data and map:**

```
python run.py
```

This script should create your synthetic dataset, train the model, output some diagnostics, and save a `risk_map.html`.

- **Start the web server:**

```
python webapp.py
```

By default, it will start on port 8001. Then open a browser to `http://localhost:8001` to see the dashboard.

- **Using the API:** You can test the API from command line or code, for example:

```
curl -X POST http://localhost:8001/score \
     -H 'Content-Type: application/json' \
     -d '{"lat":0.5,"lon":32.5,"date":"2024-01-10"}'
```

This will return a JSON with the risk score for that point. Similarly, POST a CSV file to `/score/batch` to score multiple locations at once.

Handling Real-Time Data

To use live data instead of synthetic:

- **NASA POWER (optional):** If network access is available, the app can fetch real meteorological data from NASA's POWER API. For each (lat,lon,date), query POWER for precipitation and temperature, then feed those values into the model. (This can be toggled by a flag like `USE_NASA_POWER=true`.)
- **Google Earth Engine (optional):** In a full system, one could use Google Earth Engine to pull up-to-date satellite imagery (flood, chlorophyll, etc.). For now, you can mock this by pre-downloaded data or skip it.
- **CSV upload:** In the dashboard, the CSV upload lets users supply their own real data (for example, recent rainfall measurements or local water testing results) along with locations and dates. The app then uses that real data to compute risk.

Impact and Equity

Early warning of outbreaks has huge benefits:

- **Save lives:** By alerting communities before an outbreak, health workers can distribute clean water, medicine or vaccines proactively. This can drastically reduce illnesses and deaths.

- **Optimize resources:** NGOs and governments can send water purification tablets, rehydrate salts, or set up clinics in the highest-risk areas *in advance*, instead of wasting supplies after many people are already sick.
- **Close the data gap:** Importantly, the necessary data (satellite imagery, climate forecasts, open population stats) are mostly **free and global** [6]. This means even low-income regions with no local labs or sensors can use advanced models. Using open data (e.g. EU's Copernicus satellites) removes barriers for vulnerable communities [6].

By following these steps, you build a complete prototype that fuses satellite/weather data with machine learning to score waterborne disease risk at any location and date. The result is an interactive map and API that officials can use to watch for trouble **before it starts**, saving lives and optimizing aid.

Sources: Outbreak statistics and impact are based on WHO data [2] [3]. Applications of satellite data and climate models in disease prediction are documented by NASA/NOAA [1] [5] and research groups [4] [6]. (These sources illustrate the underlying concepts; the actual code and system design above is an original step-by-step plan.)

---

[1] The Water Planet - NASA Science
https://science.nasa.gov/earth/earth-observatory/the-water-planet-46209/

[2] [3] Drinking-water
https://www.who.int/news-room/fact-sheets/detail/drinking-water

[4] [6] Mapping floods and pathways for water-borne diseases with satellites - Plymouth Marine Laboratory
https://pml.ac.uk/news/mapping-floods-and-pathways-for-water-borne-diseas/

[5] Early Warning System: Study Explores Future Risk of Waterborne Disease in a Warming Climate - NOAA/AOML
https://www.aoml.noaa.gov/early-warning-waterborne-disease/